

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Curso: Análisis Mecánico
DocIng. Oscar Maldonado
Grupo: 12

Fase 1: Investigación

Integrantes:
Marlon Ivan Carreto Rivera 201230088

Guía de Desafío:

"La Danza de los Péndulos con IA"

Fase 1: Investigación

1. Física del Péndulo: Fundamentos y Ecuaciones

El estudio del péndulo simple combina principios fundamentales de la mecánica clásica con aproximaciones matemáticas que permiten su modelado computacional.

1.1. La Ecuación de Movimiento y la Aproximación de Ángulos Pequeños

Pregunta de la guía de desafío: Física del Péndulo:

¿Cuál es la ecuación de movimiento para la posición angular $\theta(t)$ de un péndulo simple para ángulos pequeños (Pista: Involucra coseno, gravedad g , longitud L , y tiempo t)?

El movimiento de un péndulo simple está gobernado por la interacción entre la gravedad y la inercia. Cuando se separa un péndulo de su posición de equilibrio, la fuerza restauradora es la componente tangencial del peso, dada por:

$$F = -mg \sin(\theta) \quad (1)$$

Sin embargo, la ecuación diferencial que resulta de este planteamiento es no lineal y compleja de resolver analíticamente sin funciones elípticas. Para simplificar el problema y modelarlo computacionalmente en una primera aproximación, utilizamos la **aproximación de ángulos pequeños**.

Fundamento Matemático: Si el ángulo de oscilación θ es pequeño (generalmente $\theta < 10^\circ$ o 15°), podemos usar la expansión en serie de Taylor para la función seno, asumiendo que $\sin(\theta) \approx \theta$ (con θ en radianes).

Resultado: Esta simplificación linealiza el sistema, transformándolo en un Oscilador Armónico Simple (M.A.S.). Bajo esta condición, la posición angular en función del tiempo se describe mediante la ecuación:

$$\theta(t) = \theta_0 \cdot \cos(\omega t + \phi) \quad (2)$$

Donde:

- $\theta(t)$: Es el ángulo en el tiempo t .
- θ_0 : Es la amplitud inicial (ángulo máximo desde donde se suelta).
- ω : Es la frecuencia angular natural del sistema.

Contexto en el código: El script implementado utiliza esta forma simplificada asumiendo que el péndulo se suelta desde el reposo (velocidad inicial cero), implicando que la fase ϕ es cero. La frecuencia angular se define intrínsecamente por la geometría del péndulo y la gravedad:

$$\omega = \sqrt{\frac{g}{L}} \quad (3)$$

En la programación, esto se traduce directamente a la línea: `theta_t = theta0 * np.cos(w * t)`, calculando la posición exacta para cada instante de tiempo discretizado.

1.2. El Fenómeno de la Ola y la Distribución de Longitudes

Pregunta de la guía desafío El secreto de la Ola: Para que los péndulos bailen en forma de "serpiente" luego se vuelvan a juntar, sus longitudes no son aleatorias. Investigar Pendulum wave lengths formula o Como calcular longitudes para pendulum wave.

El patrón visual hipnótico observado en simulaciones de múltiples péndulos (conocido como *Pendulum Wave*) surge de la diferencia en los períodos de oscilación de cada péndulo individual. Sabemos que el periodo T (tiempo en dar una oscilación completa) es:

$$T = 2\pi\sqrt{\frac{L}{g}} \quad (4)$$

Esto implica una relación de proporcionalidad directa con la raíz cuadrada de la longitud ($T \propto \sqrt{L}$). Por lo tanto, péndulos con longitudes distintas oscilarán a frecuencias distintas, desfasándose progresivamente con el tiempo.

Tu Simulación Específica (Distribución Lineal): En experimentos estrictos de “Ondas de Péndulo”, las longitudes se ajustan para que las frecuencias sean múltiplos exactos. Sin embargo, en el código actual optamos por una distribución lineal de longitudes.

Fórmula Implementada:

$$L_i = L_{min} + \frac{(L_{max} - L_{min})}{N - 1} \cdot i \quad (5)$$

Donde N es el número de péndulos. En Python, esto se logra eficientemente con `np.linspace(1.0, 2.0, num_pendulos)`. Aunque esta distribución no garantiza una sincronización cíclica perfecta, genera un efecto de desfase progresivo estético, donde la “ola” visual parece viajar, romperse y devolverse debido a que las velocidades angulares relativas (ω) decrecen a medida que aumenta L .

2. Herramientas Computacionales en Python para Física

La implementación de simulaciones físicas requiere herramientas que permitan cálculo numérico eficiente y visualización dinámica. Python ofrece un ecosistema robusto para este fin.

2.1. NumPy: El Motor Matemático

Librería: `import numpy as np`

NumPy (*Numerical Python*) es el estándar para computación científica. Su uso en este código es fundamental debido a la **Vectorización**.

- **Por qué lo usamos:** En lugar de calcular la posición de cada péndulo individualmente dentro de un bucle `for` (computacionalmente lento en Python puro), NumPy permite operar sobre arreglos completos (*arrays*).
- **Aplicación en el código:** Cuando ejecutamos `w = np.sqrt(g / longitudes)`, NumPy calcula la raíz cuadrada para todas las longitudes simultáneamente. Del mismo modo, funciones como `np.cos()` actúan sobre todo el vector a la vez, optimizando el tiempo de ejecución.

2.2. Matplotlib: Visualización de Datos

Librería: `import matplotlib.pyplot as plt`

Matplotlib traduce los arreglos numéricos en representaciones gráficas. Para una simulación física correcta, es vital configurar el “lienzo” adecuadamente:

1. **Relación de Aspecto (Aspect Ratio):** Es crucial mantener una proporción 1:1. Sin esto, la trayectoria circular del péndulo se distorsionaría viéndose elíptica.
2. **Transformación de Coordenadas:** El código maneja la actualización constante de las coordenadas (x, y) transformadas desde coordenadas polares:

$$x = L \cdot \sin(\theta) \tag{6}$$

$$y = -L \cdot \cos(\theta) \tag{7}$$

[Image of simple pendulum free body diagram]

2.3. Animación y Renderizado en la Nube (Google Colab)

Librerías: `matplotlib.animation`, `IPython.display`

La animación física presenta un reto particular en entornos *headless* (sin monitor físico) como Google Colab.

- **Lógica del Movimiento (FuncAnimation):** Esta clase actúa como el “reloj” de la simulación, llamando repetidamente a una función de actualización (*update function*) que recalcula la física para $t + \Delta t$.
- **La Solución (IPython.display.HTML):** Dado que `plt.show()` no funciona en servidores remotos de la misma manera que en un escritorio local, debemos “renderizar” la animación convirtiéndola a HTML5/JavaScript (`to_jshtml`). Esto compila los cuadros generados y los incrusta directamente en el notebook, permitiendo una visualización fluida e interactiva.